

NPTEL • EXAM QUICK REVISION

Introduction to Large Language Models

Knowledge Graphs • PEFT • Quantization • Interpretability

Visual-first • Last-minute revision • MCQ-oriented

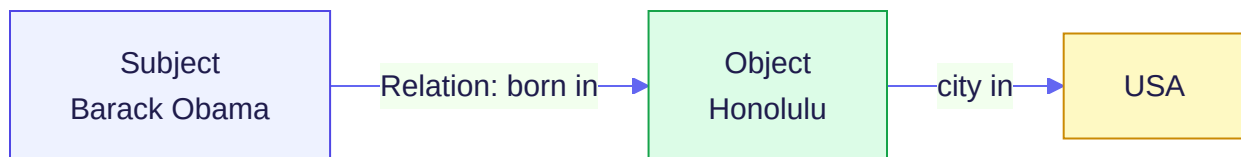
Scope of this guide (honest note)

- Built **only** from the provided Lecture 9 & 10 transcripts.
- Covers: knowledge graphs & KG completion, TranE/RotatE, transfer learning, PEFT (prompt/prefix/adapters/LoRA/QLoRA), quantization, pruning, distillation, the residual-stream view, and interpretability.
- Topics from the generic template that these lectures do **not** teach (basic tokenization, GPT-vs-BERT, RAG embeddings) are marked “not covered” instead of invented.

1. Knowledge Graph (KG) Basics

Exam definition: A knowledge graph is a network whose **nodes are entities** (e.g., Barack Obama, Honolulu) connected by **edges labelled with relation types** (e.g., *born in*). One edge = one **fact / triple** = (subject, relation, object).

In easy words: think of Wikipedia facts turned into dots and arrows. “Obama → born in → Honolulu” is one arrow. Examples of real KGs: YAGO WordNet Freebase Wikidata (largest public KG).



A KG triple: subject → relation → object, with attributes on nodes

- **Entity ID** is unique & canonical to remove ambiguity — in Wikidata, Barack Obama = **Q76** (starts with Q).
- **Relation ID** starts with **P** — “position held” = **P39**.
- **Aliases** = other names for the same entity, in many languages (Wikidata is multilingual).
- Beyond binary relations, **n-ary** relations use a diamond/relationship node (e.g., actor–role–movie).

★ Exam points

Q76 = entity ID for Obama (Q = entity); P39 = relation ID (P = property/relation). KGs are **authoritative + structured** but **incomplete** — e.g., place of birth unknown for **93.8%** of people in Freebase → motivates **KG completion**.

⚠ Common MCQ trap

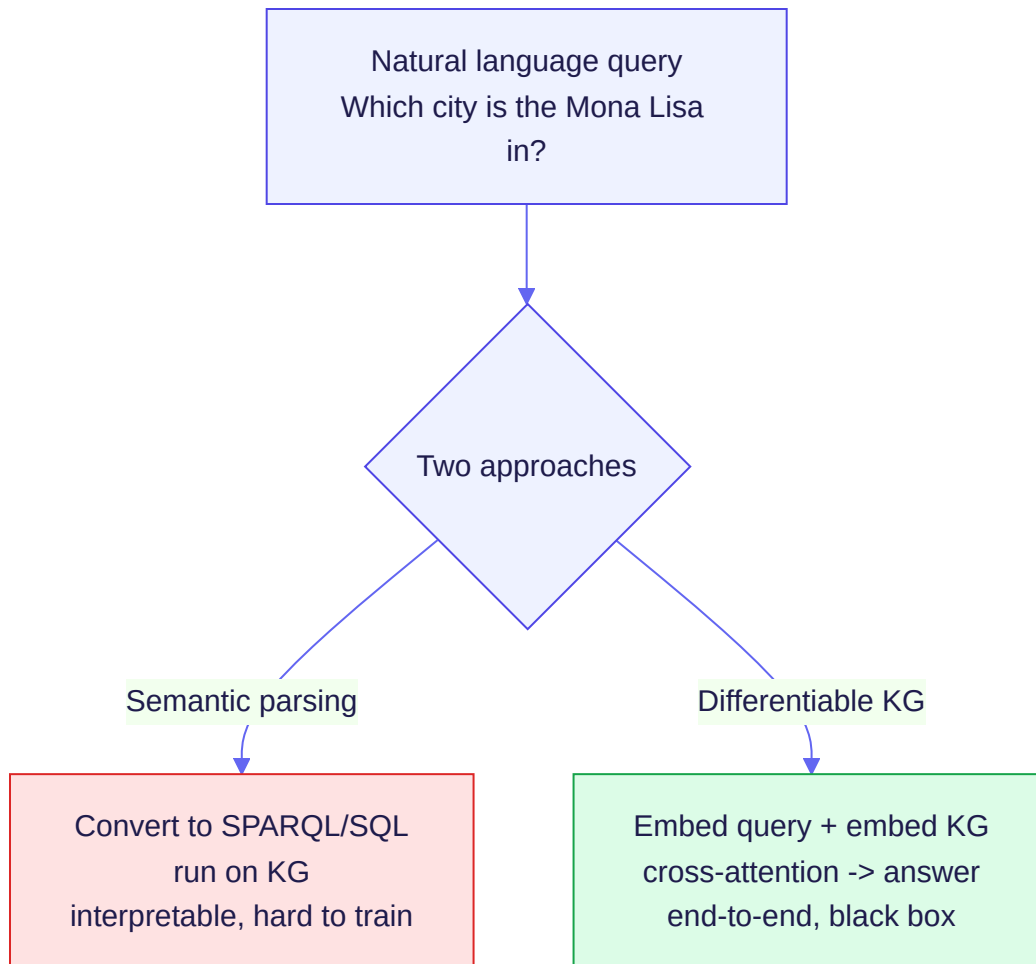
Q-number vs P-number: **Q = entity**, **P = property/relation**. Students swap them. Also: the ID string (Q76) has **no meaning** — it is just a unique opaque handle.

Term	Meaning	Exam importance
Triple / fact	(subject, relation, object)	Core definition — very high
Entity ID (Q)	Unique canonical handle for an entity	High (Q76 example)
Relation ID (P)	Unique handle for a relation	High (P39 example)
Alias	Alternative name(s), multilingual	Medium
Incompleteness	Missing facts (93.8% Freebase)	High — motivates completion

2. KG Question Answering (KGQA)

Exam definition: Answering a natural-language question by **tracing a path** through KG entities/relations. Needs (a) **disambiguation** of the entity and (b) **inference** over relations.

Example: “Which city is the Mona Lisa in?” → disambiguate “Mona Lisa” (the painting, Q12418) → exhibited at Louvre → Louvre located in Paris → Paris is a city → answer **Paris**. The LLM supplies world knowledge for disambiguation; the KG supplies the current, changeable facts.



Two approaches to KGQA: semantic parsing vs differentiable KG embedding

Approach	How	Pro	Con
Semantic parsing / interpretation	NL → structured query (SQL / SPARQL) run on the graph	Interpretable for experts	Hard to implement, not end-to-end trainable
Differentiable / end-to-end KG	Embed query (BERT/BART) + embed KG (GCN/GNN/graph transformer), cross-attention selects answer subgraph	End-to-end trainable, no formal query needed	Black-box, hard to debug

★ **Why put LLMs + KGs together?**

1. Facts change (presidents, CEOs) — KG edits are cheap, retraining an LLM is expensive. **2.** Isolates mutable factual knowledge from LLM reasoning. **3. Guards against hallucination** (consult KG instead of sampling made-up facts). **4.** Improves interpretability (see which KG regions were consulted).

⚠ Trap

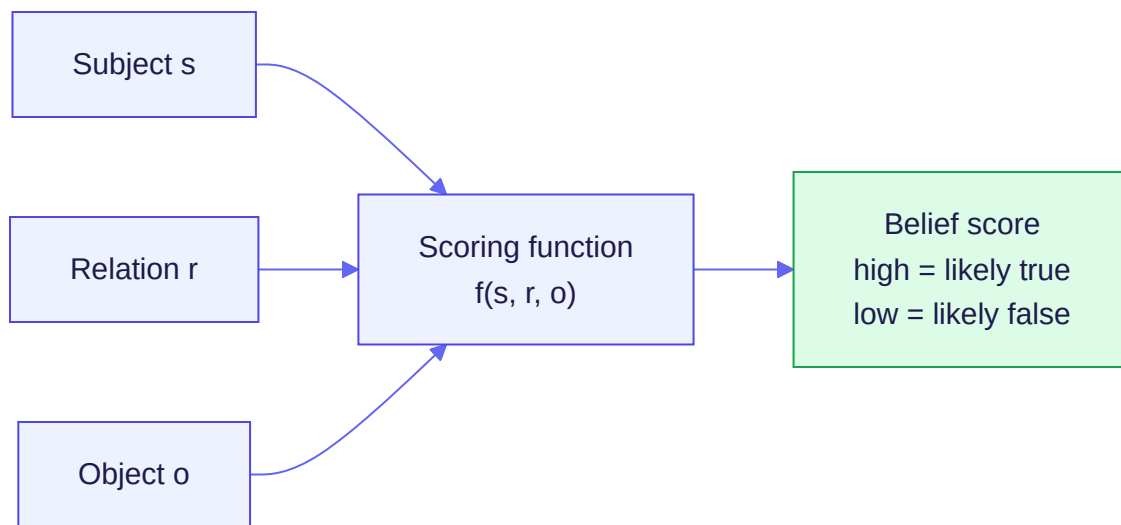
ChatGPT answering famous-entity questions does NOT make KGs unnecessary — coverage drops sharply for **torso / long-tail entities** and low-resource languages, where the LLM–KG synergy matters.

💡 1-Minute Revision Sheet — KG Basics & QA

- Triple = (subject, relation, object). Q = entity ID, P = relation ID.
- KGs are authoritative but **incomplete** (Freebase 93.8% missing birthplaces).
- KGQA = disambiguate + trace a path. Two ways: **semantic parsing** (SPARQL, interpretable) vs **differentiable embedding** (end-to-end, black-box).
- LLM+KG combo → up-to-date facts, less hallucination, more interpretability.

3. KG Completion & Evaluation

Exam definition: Predicting missing triples using a learned **scoring function** $f(s, r, o)$ that outputs **belief** that the triple is true. Like **link prediction** in a social network, but with rich relation/entity features.



KG completion: a scoring function maps (s, r, o) to a belief score

Training the scorer

- Give **positive triples** (in KG) and **negative triples** (sampled). Negatives made by **corrupting** one slot (replace s or o with a random entity) — the **local closed-world assumption**.
- Turn score into probability with a **softmax** over all candidate objects; then use **negative log-likelihood** loss. Denominator is expensive → use **negative sampling** (K samples, rescale by E/K).
- Alternative: **margin / hinge (ReLU) loss** — good score must beat bad score by a margin (like an SVM). Perturb **one slot at a time** to limit the search space.

💡 Memory shortcut

Polarity: some models use **score** (higher = truer); distance-based models (TransE) use **distance** (smaller = truer). Flip the loss sign accordingly.

Evaluation metrics

Metric	Meaning	When
Hits@K	Fraction of correct answers appearing in top-K	Multiple correct answers (1-to-many)
MRR (mean reciprocal rank)	$1 / (\text{rank of first correct answer})$	Exactly one correct answer
MAP	Mean average precision across relevant items	Several relevant items
Filtered eval	Remove already-known-true triples from the ranked list before scoring	Fair comparison

★ MRR arithmetic (very common numeric MCQ)

First correct answer at rank 2 $\rightarrow RR = 1/2 = 0.5$. At rank 1 $\rightarrow 1.0$ (max). MRR is “mean” = it penalises rank 1 $\rightarrow$ 2 as harshly as rank 2 $\rightarrow$ infinity. Both Hits@K and MRR range **0 to 1**.

⚠ Trap: filtered vs unfiltered

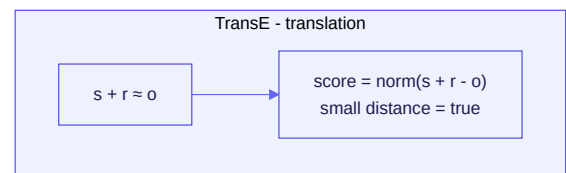
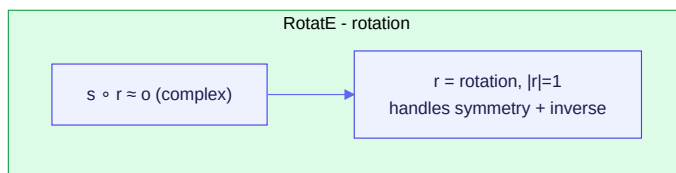
Papers report different numbers depending on whether they filtered out known-true triples. If a known-true triple O2 sits above your gold answer, filtered MRR removes it $\rightarrow$ rank improves (e.g., $1/6 \rightarrow 1/5$). Always check which was used.

Standard datasets

Dataset	Source	Notes
WN18 / WN18RR	WordNet (words/synsets)	18 relation types; RR = harder, fixes leakage
FB15K / FB15K-237	Freebase (15k entities)	1,345 relations; 237 = leakage-fixed variant
YAGO	Ontology	Another standard benchmark

WN18 train/dev/test $\approx 141k / 5k / 5k$ triples; FB15K train $\approx 480k$. WN18RR & FB15K-237 were created because the originals let models “cheat” via inverse relations.

4. KG Representation Models: TransE & RotatE



Translation (TransE) vs rotation (RotatE) scoring

TransE (translation)

Idea borrowed from word2vec analogies (king – man + woman $\approx$ queen). Relation = a **translation vector**: $\mathbf{s} + \mathbf{r} \approx \mathbf{o}$.
Score = $\|\mathbf{s} + \mathbf{r} - \mathbf{o}\|$ (small distance = high belief).

- **Pros**: extremely simple & fast, almost no parameters in f , few hyperparameters (just margin + #negative samples).
- **Cons**: cannot model **1-to-many** / **many-to-1** / **many-to-many** (forces multiple objects to collapse to the same point); cannot model **symmetric** relations (would need $\mathbf{r} = \mathbf{0}$).

RotatE (rotation)

Entities = **complex vectors**; relation = **rotation** in the complex plane with $|\mathbf{r}| = 1$ per dimension: $\mathbf{s} \circ \mathbf{r} \approx \mathbf{o}$. Multiplying by a unit complex number rotates by angle θ .

- Handles **symmetry** (rotation is its own inverse $\rightarrow \theta = 180^\circ$), **anti-symmetry**, **inversion** (clockwise vs anticlockwise), and **composition** (add angles).
- **Superset of TransE** — at least as powerful.
- Best accuracy usually from **factorization** / **multiplicative** models (tensor factorization).

⚠ Classic exam trap

TransE's headline weakness: **symmetric** relations and **1-to-many** relations. RotatE fixes symmetry because a rotation can be its own inverse. TransH (hyperplane projection) fixes many-to-many by projecting onto a relation-specific hyperplane.

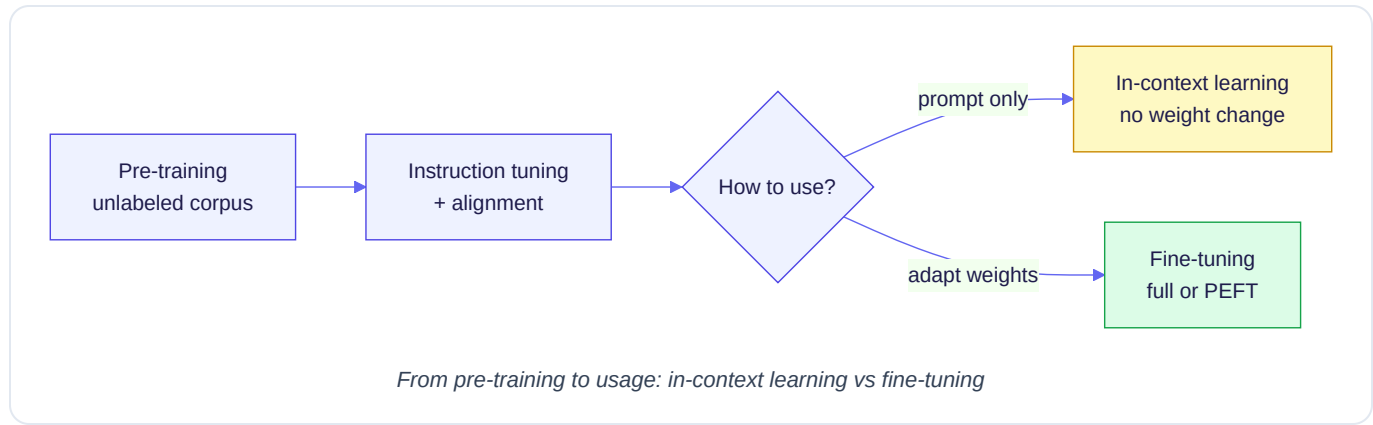
💡 Mnemonic

“**Trans** = **add** (translate), **Rot** = **spin** (rotate).” Distance small = good for TransE; rotation with unit modulus = RotatE.

🕒 1-Minute Revision Sheet — Completion & Models

- $f(\mathbf{s}, \mathbf{r}, \mathbf{o}) \rightarrow$ belief. Negatives via corruption (local closed-world assumption).
- Losses: softmax + NLL (needs negative sampling) OR margin/hinge (SVM-like).
- Hits@K (multiple answers), MRR (one answer, $= 1/\text{rank}$), filtered vs unfiltered matters.
- TransE: $\mathbf{s} + \mathbf{r} \approx \mathbf{o}$, fast, fails on symmetric & many-to-many. RotatE: rotation, $|\mathbf{r}|=1$, superset of TransE.
- Datasets: WN18(RR), FB15K(-237), YAGO. The -RR/-237 versions fix inverse-relation leakage.

5. Transfer Learning: ICL vs Full Fine-Tuning



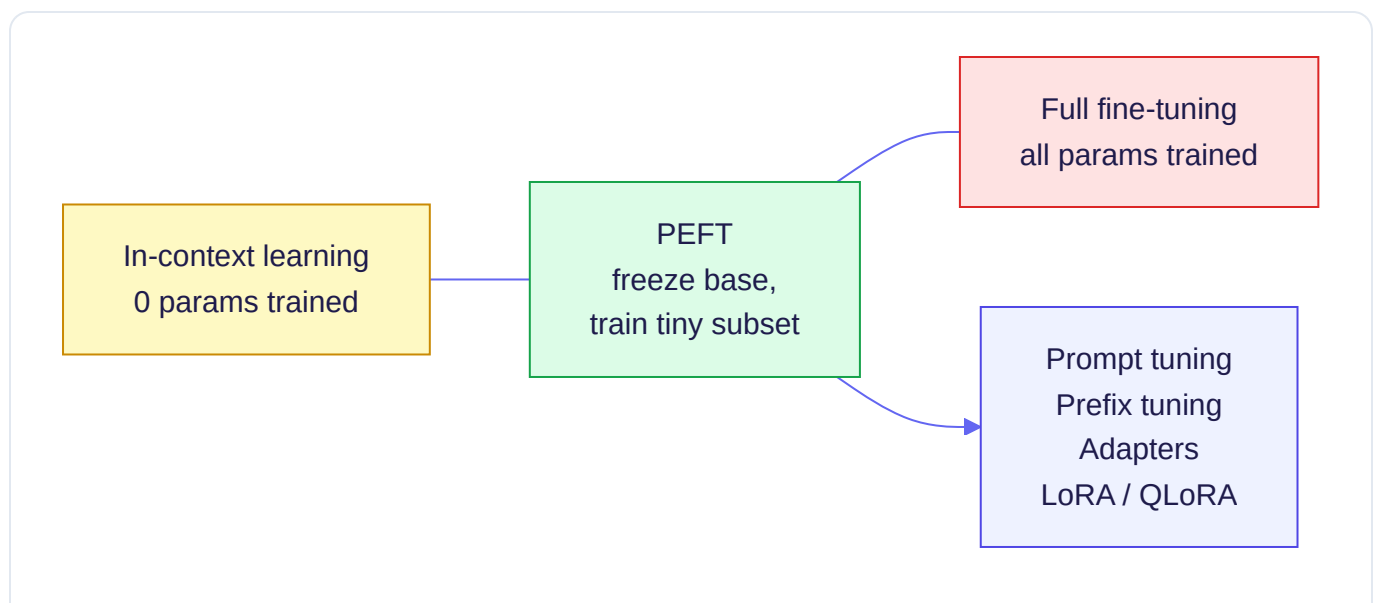
In-context learning (ICL) = “hard prompting”: feed task info (instructions + examples) into the prompt; **no weights change**. Cheap, uses a hosted LLM via API.

Why ICL is not enough

- Lower accuracy than full fine-tuning on critical tasks.
- **Prompt sensitivity** — order of examples, a missing word can swing accuracy. New model version (Llama 3 → 3.1) → redo everything.
- Prompt tokens cost **latency + throughput + energy**; long instructions eat the context window.

Why NOT full fine-tuning

- **Memory:** need ~**12–20×** the parameter memory (weights + optimizer states + gradients + activations). An 8B model in FP16 ≈ 16 GB just for weights; GPT-3 175B is infeasible for most.
- **Storage:** a full checkpoint (e.g., 350 GB) **per task** is unmanageable to save and serve.
- **Overfitting:** huge model + small data → memorises; needs enormous data to avoid.



PEFT (Parameter-Efficient Fine-Tuning): freeze most of the network, train only a **tiny subset** of parameters. Gets fine-tuning-level accuracy with a fraction of the cost.

★ **PEFT benefits (frequently asked list)**

Less memory (fewer optimizer states/gradients) → fewer/older GPUs (V100 not just A100/H100); faster convergence; less **overfitting**; resists **catastrophic forgetting** (keeps world knowledge, generalises across domains); tiny **storage** (save only incremental weights).

⚠ **Trap: catastrophic forgetting**

Learning task B can erase task A in normal fine-tuning. PEFT largely avoids this because the base (world knowledge) stays frozen.

6. The Four PEFT Techniques

(a) Prompt tuning / soft prompting

Learn a few **soft-prompt tokens** (trainable embeddings) prepended to the input; **everything else frozen**. Only trainable params = these tokens.

- **Soft** (model learns embeddings) vs **hard** (human edits words). Save just the tiny prompt per task.
- Big win = **multi-task serving**: one frozen base LLM, swap task-specific soft-prompt tokens per batch.
- Findings: catches up to full fine-tuning **as model size grows**; gains **plateau ~20 prompt tokens** (T5); good **initialization** (from real word embeddings) matters; generalises better out-of-domain (e.g., SQuAD → books).

(b) Prefix tuning

Like prompt tuning but adds trainable parameters at **every transformer layer**, not just the input embedding.

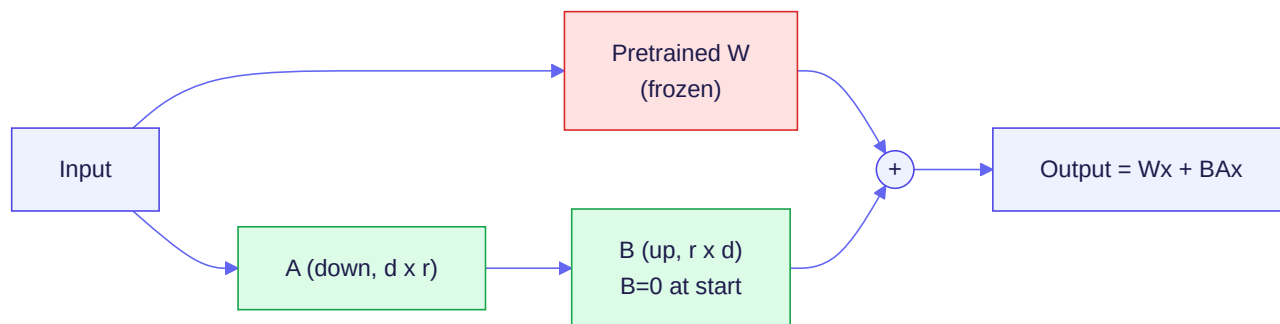
- Reparameterised through a small MLP ($f\theta'$) to avoid **unstable training**; MLP can be merged away after training.
- ~**0.1%** of parameters reaches full-fine-tuning-comparable results; works on unseen domains.

(c) Adapters

Insert small **adapter layers** inside each transformer block: **down-project** → **nonlinearity** → **up-project** (a bottleneck), with a residual connection.

- First popular PEFT (2019). Bottleneck dimension m is the accuracy/overfitting knob.
- Reaches full-fine-tuning performance with ~**3.6%** params.
- **Downside:** adds layers **in series** → **inference latency overhead**; hard to hot-swap. (Prompt/prefix/LoRA avoid this.)

(d) LoRA (Low-Rank Adaptation)



LoRA: frozen W in parallel with a low-rank $B \cdot A$ update (B starts at 0)

Represent the weight update as a **low-rank product**: $\Delta W = B \cdot A$ (down-project A , up-project B , **no nonlinearity**), added in parallel to frozen W . Output = $Wx + BAx$.

- Grounded in **intrinsic dimensionality**: a task needs far fewer effective dimensions than the full parameter count; capable base model + easy task $\rightarrow$ tiny rank r (even $r=1$).
- Apply to the **query, key, value, output** projection matrices. **Rank r** = accuracy knob ($\approx$ intrinsic dim).
- **Init**: $B = 0$ (so $BA = 0$ at start $\rightarrow$ model unchanged), A = small Gaussian $\rightarrow$ stable training.
- **No nonlinearity** so BA can be **fused into W** after training (no inference overhead).
- Variants: **QLoRA** (quantized, memory-saving), LongLoRA, LoRA+ (diff learning rates for A, B), DyLoRA (auto-rank).

💡 Memory shortcut

“**LoRA = Low-rank, no nonlinearity, fuse-able. Adapter = bottleneck WITH nonlinearity, in-series, adds latency.**” B starts at 0, A starts small-Gaussian.

⚠️ Trap

Adapter vs LoRA both use a bottleneck — but adapters have a **nonlinearity** and add inference latency; LoRA has **no nonlinearity** and fuses into W (zero added latency). If forced to choose, prefer LoRA/prefix over adapters.

Comparison table — PEFT family

Technique	What is trained	Where	Approx params	Inference latency
Prompt tuning	Soft-prompt embeddings	Input layer only	Tiny (few tokens)	None added
Prefix tuning	Prefix params + MLP	Every layer	$\sim 0.1\%$	None added
Adapters	Bottleneck FFN layers	Inside each block (series)	$\sim 3.6\%$	Adds latency
LoRA	Low-rank $B \cdot A$ on $Q/K/V/O$	Parallel to weights	Very small	None (fused)

ICL vs Full Fine-Tuning vs PEFT

	In-context learning	Full fine-tuning	PEFT
Weights changed	None	All	Tiny subset
Memory	Lowest	Highest (12–20 $\times$)	Low
Storage/task	None	Full checkpoint	Tiny increment

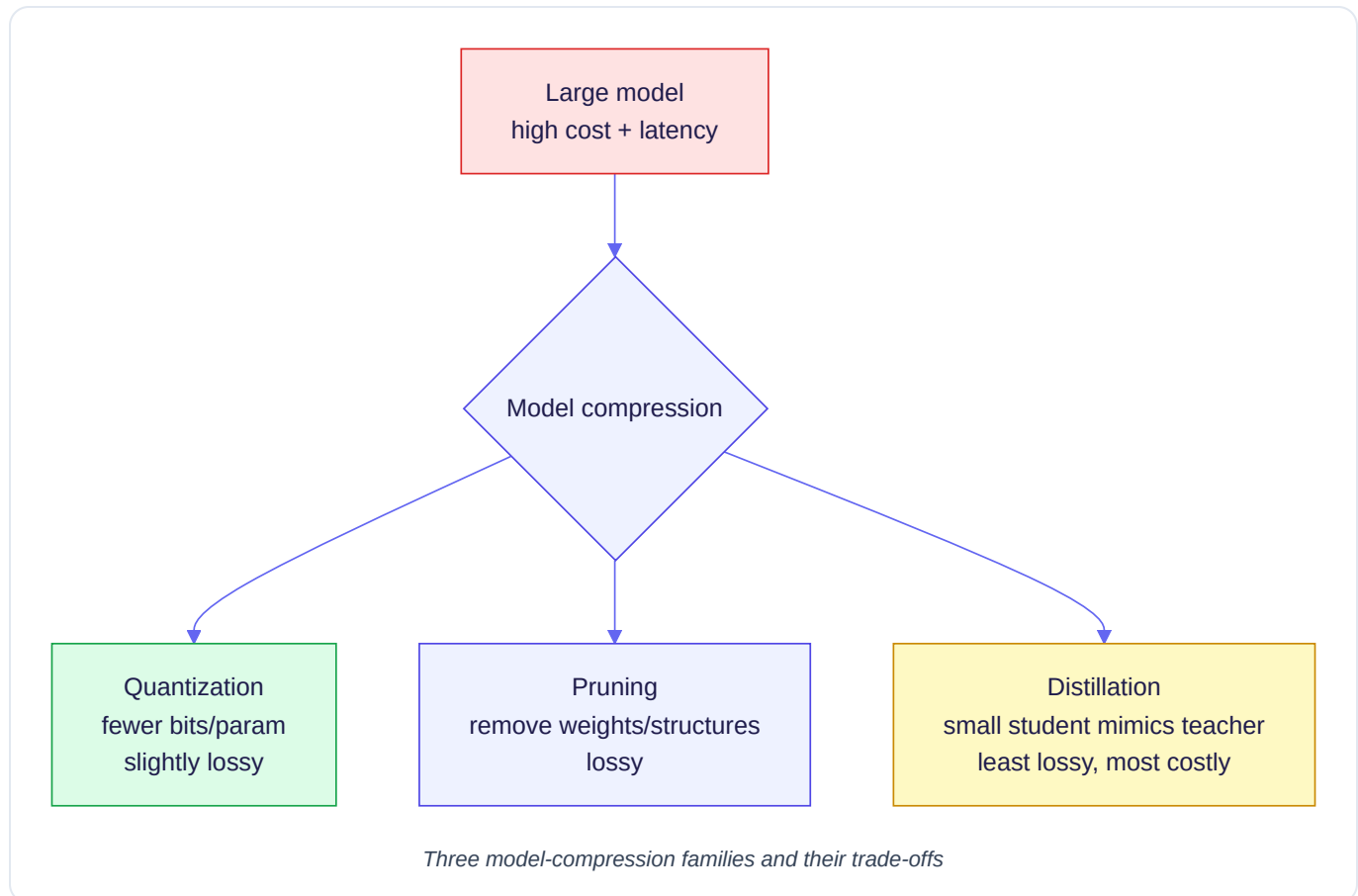
Accuracy	Lower	Highest	Near full FT
Forgetting	None	Prone	Resistant

🕒 1-Minute Revision Sheet — PEFT

- Full FT needs 12–20× param memory + huge per-task storage → PEFT freezes base, trains a tiny subset.
- Prompt tuning = input soft tokens (plateau ~20, great for multi-task serving). Prefix = every layer.
- Adapters = bottleneck + nonlinearity in series (adds latency, ~3.6%). LoRA = low-rank BA, no nonlinearity, fuses (0 latency).
- LoRA init: B=0, A=Gaussian. Apply to Q/K/V/O. Rank $r \approx$ intrinsic dimensionality. QLoRA = quantized LoRA.

7. Why compress? Inference cost

Big models → new hardware, high **latency**, high **inference cost**, **sustainability/energy** concerns. Example: GPT-4o mini is far cheaper than GPT-3.5-Turbo for a small accuracy drop — often worth it commercially.



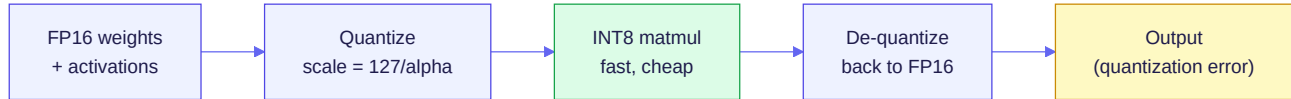
Technique	Idea	Lossy?	Cost
Quantization	Fewer bits per weight/activation	Slightly	Cheap, few changes
Pruning	Remove weights / structures	Yes	Cheap-ish
Distillation	Small student mimics big teacher	Least lossy	Most expensive (retrain + data)

8. Quantization

Exam definition: Represent parameters/activations with **fewer bits** (e.g., FP32 → FP16 → INT8) to save memory and speed up matmuls.

Format	Bits (sign/exp/mantissa)	Note
--------	--------------------------	------

FP32 (full)	1 / 8 / 23	High precision, scientific default
FP16 (half)	1 / 5 / 10	Deep-learning default, GPU support
BF16 (brain float)	1 / 8 / 7	Keeps FP32 range , less precision; FP32 → BF16 = simple truncation
INT8	1 sign / 7	Symmetric quantization example



Symmetric quantization: FP16 → INT8 matmul → de-quantize (with error)

★ Scale factor (numeric MCQ)

Symmetric INT8: find max-abs value α in the vector, map $[-\alpha, \alpha]$ to $[-127, 127]$. **scale = $127 / \alpha$** . Quantize: value $\times$ scale, then round. De-quantize: quantized $\div$ scale. The gap between original and de-quantized = **quantization error**. Each vector has its own α /scale.

Two usage modes

- **Post-Training Quantization (PTQ)**: quantize after training, cheap, no retraining. Weights fixed; for inputs/activations use a small **calibration dataset** to estimate scale factors.
- **Quantization-Aware Training (QAT)**: quantization simulated during training (e.g., QLoRA).
- **Symmetric** (0 → 0) vs **asymmetric** (0 not mapped to 0, extra zero-point constant).

⚠ Outliers & LLM.int8()

Beyond **~2.7B** params, a few **outlier** activation values wreck INT8 (one huge value forces the scale so everything else collapses to ~0). Fix (**LLM.int8()**): keep outlier columns/rows in FP16, quantize the rest to INT8, then recombine. Contribution = **finding the problem**; the fix is simple.

QLoRA (quantization-aware fine-tuning)

- Goal: cut memory to do LoRA. A 65B model full FT $\approx$ 780 GB; QLoRA fits in **~48 GB** with almost no quality loss.
- Four pieces: **4-bit NormalFloat (NF4)** (buckets assume a normal distribution, not uniform), **double quantization** (quantize the quantization constants too — saves storage, not accuracy), **paged optimizers** (page between GPU ↔ CPU memory), and **LoRA** on top. Compute in **BF16**.

💡 Shortcut

“NF4 = better buckets (accuracy). Double-quant = smaller constants (storage). Paging = don’t run out of GPU memory.”

9. Pruning

Exam definition: Remove parts of the network while keeping performance.

Type	Idea	Catch
Unstructured (magnitude pruning)	Drop lowest-magnitude weights (~40% often safe; fine-tune to recover up to ~80%)	No speed-up unless hardware exploits sparsity (a stored 0 still costs FP16)

Structured	Remove whole heads / layers / hidden dims / use 2:4 sparsity (A100)	Hardware-friendly → real speed-up
-------------------	--	-----------------------------------

⚠ Trap

Unstructured pruning alone gives **no real speed-up** unless the hardware supports sparsity — that's why the field moved to **structured** pruning (e.g., 2:4 pattern). Also: consider **weights × activations**, not weights alone.

10. Knowledge Distillation

Exam definition: Train a small **student** to imitate a large **teacher** — match the teacher's output **distribution**, not just the hard label.

- **Least lossy** compression but **most expensive** (run teacher inference over lots of data, then train student).
- Biggest bottleneck = **data** (models are open-sourced, data usually isn't) → often **task-specific** distillation.
- **Soft targets** (full distribution) beat **hard targets** (argmax=1). Losses: cross-entropy, KL / reverse-KL divergence.
- **Sequence-level** distillation approximates the huge output space by the beam-search top sequence.
- Modern twist: a giant model (e.g., 405B) generates **synthetic data** to fine-tune a small model — e.g., **Self-Instruct** (start from ~175 seed tasks, ask the LLM to generate more).

💡 Shortcut

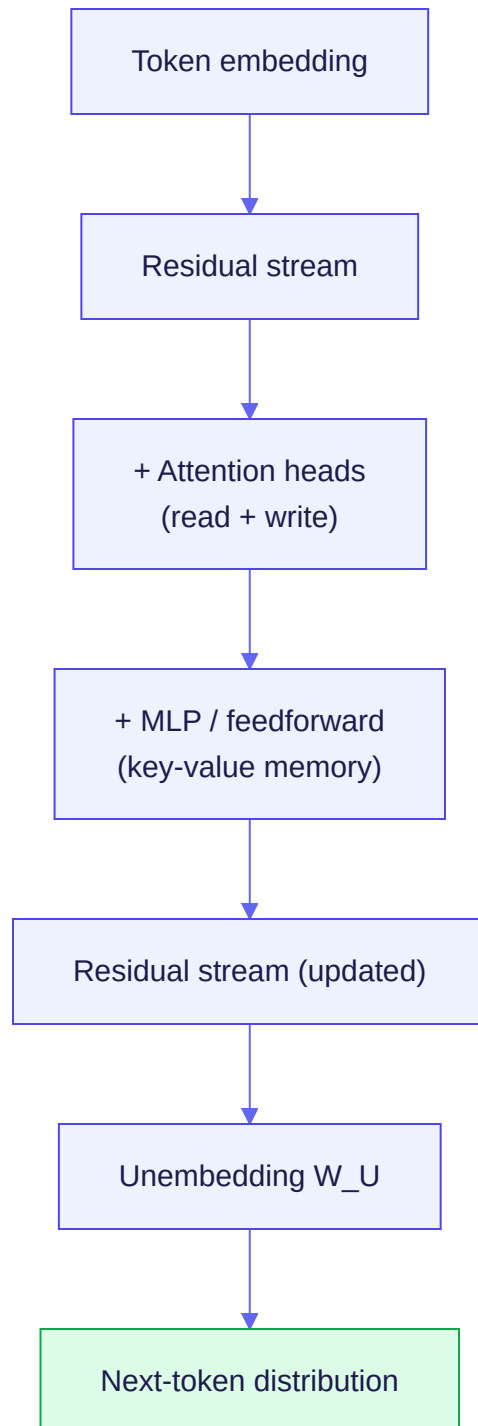
"Quantize = same model, fewer bits. Prune = cut pieces. Distil = small student copies big teacher." Distillation needs data + is costly but keeps the most accuracy & gives the biggest speed-up.

🕒 1-Minute Revision Sheet — Compression

- Quantize: scale = $127/\alpha$ (INT8). PTQ (calibration set) vs QAT (QLoRA). BF16 keeps FP32 range.
- Outliers break INT8 above ~2.7B → LLM.int8() keeps outliers in FP16.
- QLoRA = NF4 + double quantization + paged optimizers + LoRA (compute in BF16), 65B in ~48 GB.
- Pruning: unstructured (no speed-up w/o HW sparsity) vs structured (2:4, real speed-up).
- Distillation: student mimics teacher (soft targets > hard); least lossy, most costly, data-hungry.

11. Residual-Stream View of a Transformer

Exam definition: View each layer's attention and MLP outputs as **additive** contributions to a shared **residual stream** (a running vector), starting from the token embedding and ending at the **unembedding** (W_U) that produces the next-token distribution.



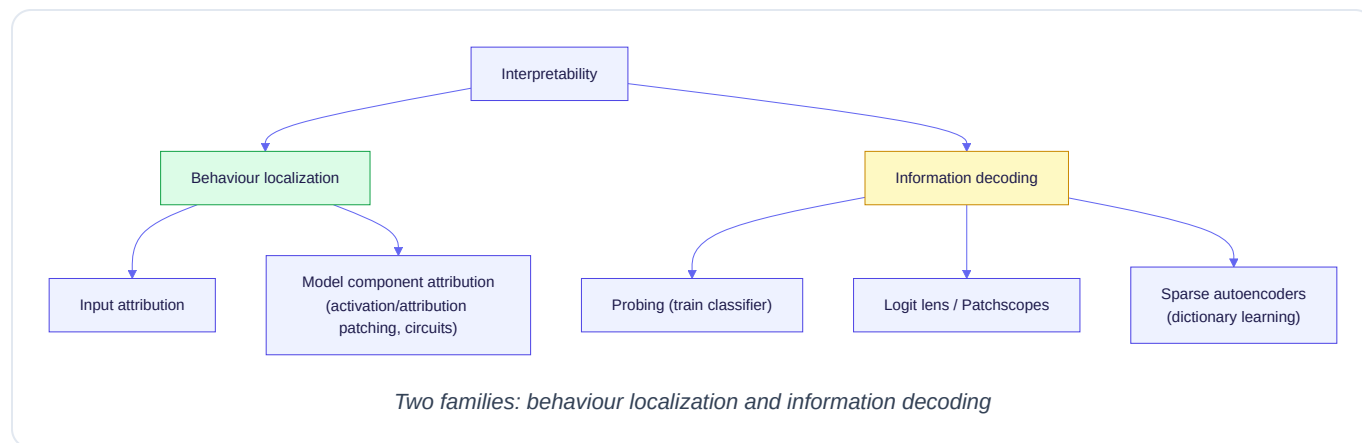
The residual stream: attention and MLP write additively; unembedding reads out

- Attention has two “circuits”: **Query-Key (QK)** circuit = **where to attend**; **Output-Value (OV)** circuit = **what to write** to the stream.
- The **MLP** acts as **key-value memory**: W_{in} detects patterns (keys), W_{out} writes values; each intermediate unit is a **neuron**.
- Decomposing gives: **direct path** (embed $\rightarrow$ unembed), **full OV circuits**, and **virtual attention heads** (V-composition = two OV circuits acting as one).

★ Exam points

Embedding layer: token $\rightarrow$ vector; **Unembedding W_U** : vector $\rightarrow$ vocabulary distribution. QK circuit = attention pattern; OV circuit = writes to residual stream. This decomposition is the basis of mechanistic interpretability.

12. Interpretability: Map of Techniques



Two broad families: (1) **Behaviour localization** — which component causes an output (input attribution, component attribution, circuits); (2) **Information decoding** — what information a hidden state encodes (probing, logit lens, sparse autoencoders).

Probing

Freeze the model; train a **classifier** on a layer's hidden states to predict a linguistic property (e.g., part-of-speech). Tests **mutual information** between hidden state and property.

⚠ Trap: probe complexity & control tasks

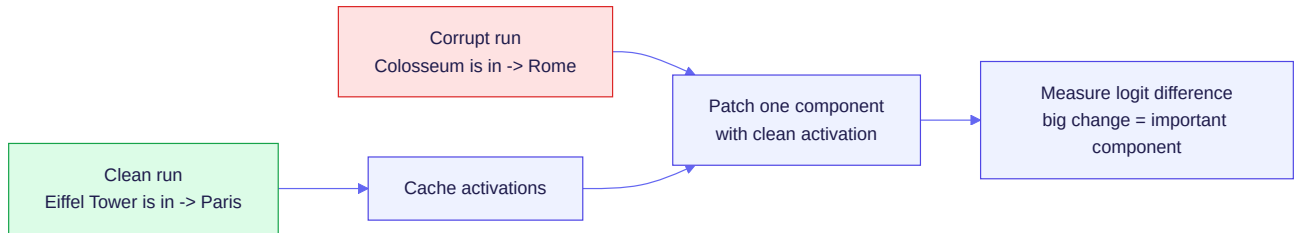
A complex probe may score high by **memorising**, not because the info is there. Fix: a **control task** (random labels). **Selectivity** = probe-task accuracy – control-task accuracy; want it **high**.

Mechanistic interpretability

Attempt to **reverse-engineer** the algorithms a network implements into human-understandable mechanisms. Key unit borrowed from vision (Chris Olah): the **circuit** = a computational **subgraph** (neurons = nodes, weights = edges) responsible for a task.

- **Polysemantic neurons:** one neuron encodes multiple concepts (car windows + body + wheels) $\rightarrow$ hard to interpret.
- Three claims: **features** (fundamental units), **circuits** (features joined by weights), **universality** (recur across models — ambitious/contested).
- Example task: **IOI** (Indirect Object Identification) — “John and Mary... John gave a drink to ___” $\rightarrow$ predict **Mary**.

Activation patching vs Attribution patching



Activation patching: swap a component's activation from a clean run into a corrupt run

	Activation patching	Attribution patching
Idea	Overwrite one component's activation with the clean value, re-run, measure logit change	Linearly approximate that change via a Taylor/gradient step
Cost	2 forward passes + one forward pass per component (expensive)	2 forward passes + 1 backprop (scalable)
Limits	Accurate	Only for single-token change & fine granularity (small Δ)

★ Clean vs corrupt run

Clean: "The Eiffel Tower is in" → Paris. Corrupt: replace with "Colosseum" → Rome. Patch a component's clean activation into the corrupt run; big **logit-difference** change = important component. **ACDC** automates circuit discovery (uses KL divergence or logit diff, prunes edges in reverse topological order).

Induction heads & in-context learning

An **induction head** is a 2-head circuit doing pattern-copying: given ...[A][B]...[A], predict [B]. Head 1 looks at the previous token; head 2 copies what followed it last time.

- Believed to be a primary driver of **in-context learning**. Needs ≥ 2 layers (a 1-layer model can't form it).
- ICL score = loss(500th token) – loss(50th token); induction heads **emerge** at the same training step ICL loss drops.
- Measured via **prefix-matching score**.

Logit lens, Patchscopes & Sparse Autoencoders (SAEs)

- **Logit lens**: multiply an intermediate residual state by W_U to see which tokens are being pushed at each layer (a special case of Patchscopes when $M=M'$ and layer = last).
- **Patchscopes**: generalised probing — patch an activation from model M into a prompt in model M' and read the answer; no classifier training.
- **SAEs / dictionary learning**: tackle **superposition** (model packs more features than dimensions). Project activations up into a **higher-dim sparse** space (encoder), reconstruct (decoder); train with **reconstruction loss + L1 sparsity penalty**. Aligns features to basis → **monosemantic**, interpretable, and steerable (Claude 3 Sonnet feature-steering, e.g., a gender-bias feature).

⚠ Traps

Polysemantic = one neuron, many concepts; **superposition** = more features than dimensions (why SAEs go **wider**, e.g., 512 → ~131k). **Linear representation hypothesis**: features are linear directions. SAE training is **unsupervised**.

💡 Shortcut

“Probing = train a classifier. Logit lens = read the stream through W_U . Patching = intervene & measure. SAE = go wide + sparse to un-mix neurons.”

🕒 1-Minute Revision Sheet — Interpretability

- Residual stream: attention + MLP add to it; QK = where to attend, OV = what to write; W_U reads out.
- Probing (classifier + control task → selectivity). Mechanistic = reverse-engineer circuits (subgraphs).
- Activation patching (per-component forward passes) vs attribution patching (1 backprop, scalable).
- Induction heads (2 heads, pattern copy) drive ICL; need ≥ 2 layers.
- Superposition → SAEs (wide + sparse, L1 + reconstruction) give monosemantic, steerable features.

Formula Sheet (only what the lectures use)

Name	Formula	Meaning
TransE score	$f = s + r - o $	Small distance = true triple
RotatE	$o \approx s \circ r$, with $ r = 1$	Relation = unit rotation (complex)
Reciprocal rank	$RR = 1 / (\text{rank of first correct})$	MRR = mean of RR over queries
Hits@K	$\# \text{correct in top-}K \div K$	Range 0–1
Softmax prob	$P(o s,r) = e^{f(s,r,o)} / \sum_{o'} e^{f(s,r,o')}$	Turns score into probability
Neg-sampling rescale	multiply sampled denom by E/K	K samples out of E entities
INT8 scale	$\text{scale} = 127 / \alpha$ ($\alpha = \max \text{value} $)	Quantize: $\text{round}(\text{value} \times \text{scale})$
De-quantize	$\text{value} \approx \text{quantized} / \text{scale}$	Difference = quantization error
LoRA	$y = Wx + B \cdot A \cdot x$, rank $r \ll d$	$B=0$, A =Gaussian at init
Full-FT memory	$\approx 12\text{--}20 \times \# \text{params}$	weights+optimizer+grads+activations
ICL score	$\text{loss@500th} - \text{loss@50th token}$	Induction / in-context learning signal

Definition Sheet (memorise these lines)

KG: entities (nodes) + relations (edges); fact = triple (s,r,o) .

KG completion: learn $f(s,r,o) \rightarrow$ belief to predict missing triples.

MRR: mean of $1/(\text{rank of first correct answer})$.

TransE: $s+r \approx o$; fails on symmetric & many-to-many.

RotatE: relation = unit-modulus rotation; superset of TransE.

ICL: task via prompt only; no weight update (hard prompting).

PEFT: freeze base, train a tiny parameter subset.

Prompt tuning: trainable soft-prompt tokens at input only.

Prefix tuning: trainable params at every layer.

Adapter: bottleneck FFN + nonlinearity in series (adds latency).

LoRA: $\Delta W = BA$ (low-rank, no nonlinearity, fuses into W).

QLoRA: NF4 + double-quant + paged optimizers + LoRA.

Quantization: fewer bits per weight/activation.

PTQ vs QAT: after training (calibration) vs during

training.

Pruning: remove weights (unstructured) or structures.

Distillation: small student mimics teacher's distribution.

Residual stream: additive per-layer vector; W_U reads out.

QK vs OV circuit: where to attend vs what to write.

Probing: train classifier on frozen hidden states.

Circuit: computational subgraph doing a task.

Activation patching: swap clean activation into corrupt run, measure logit diff.

Induction head: 2-head pattern-copy circuit; drives ICL.

Superposition: more features than dimensions → SAEs un-mix them.

Polysemantic neuron: one neuron, many concepts.

Not covered in these lectures (so no invented content): basic tokenization mechanics, GPT-vs-BERT architecture comparison, prompt-engineering how-to, and RAG embedding/context-window internals. The generic template lists them, but Lectures 9–10 do not teach them. RAG appears only conceptually (KGs reduce hallucination like retrieval does).

Top MCQs (with answers) — grounded in the lectures

59 high-probability MCQs. Answers highlighted in green. These test exactly the facts, numbers, and traps emphasised in Lectures 9–10.

Q1. In a KG triple (s, r, o), what does the object represent?

- A. The relation label
- B. The entity the relation points to
- C. The unique entity ID
- D. The graph schema

Answer: B. The entity the relation points to

Q2. In Wikidata, an entity ID starts with which letter?

- A. P
- B. R
- C. Q
- D. E

Answer: C. Q

Q3. A relation/property ID in Wikidata starts with:

- A. Q
- B. P
- C. R
- D. W

Answer: B. P

Q4. Roughly what fraction of people in Freebase have an unknown place of birth (per lecture)?

- A. 9.38%
- B. 38.9%
- C. 93.8%
- D. 3.98%

Answer: C. 93.8%

Q5. Which KGQA approach is end-to-end trainable but hard to debug?

- A. Semantic parsing to SPARQL
- B. Differentiable KG embedding + cross-attention
- C. Manual SQL
- D. Rule-based lookup

Answer: B. Differentiable KG embedding + cross-attention

Q6. Which KGQA approach is most interpretable to experts?

- A. Differentiable embedding
- B. Graph neural network
- C. Semantic parsing to SPARQL/SQL
- D. Cross-attention

Answer: C. Semantic parsing to SPARQL/SQL

Q7. Corrupting the subject of a positive triple to create a negative is based on the:

- A. Open-world assumption
- B. Local closed-world assumption
- C. Global truth assumption
- D. Uniform prior

Answer: B. Local closed-world assumption

Q8. If the first correct answer appears at rank 4, the reciprocal rank is:

- A. 4
- B. 0.25
- C. 0.75
- D. 1.0

Answer: B. 0.25

Q9. Which metric is best when there is exactly ONE correct answer?

- A. Hits@K
- B. MRR
- C. Precision
- D. Recall

Answer: B. MRR

Q10. TransE scores a triple using:

- A. Rotation angle
- B. $\|s + r - o\|$
- C. Dot product $s \cdot o$
- D. $\text{Cosine}(s, r)$

Answer: B. $\|s + r - o\|$

Q11. TransE fundamentally cannot model which relation type well?

- A. 1-to-1
- B. Symmetric relations
- C. Sparse relations
- D. Directed relations

Answer: B. Symmetric relations

Q12. RotatE represents a relation as a:

- A. Translation vector
- B. Hyperplane
- C. Unit-modulus rotation in complex space
- D. Diagonal matrix of integers

Answer: C. Unit-modulus rotation in complex space

Q13. Why were WN18RR and FB15K-237 created?

- A. To add more entities
- B. To fix inverse-relation leakage that let models cheat
- C. To reduce dataset size for speed
- D. To add images

Answer: B. To fix inverse-relation leakage that let models cheat

Q14. In-context learning changes how many model weights?

- A. All of them
- B. A small subset
- C. None
- D. Only the embedding layer

Answer: C. None

Q15. Approximate memory needed for FULL fine-tuning relative to #parameters?

- A. 1-2x
- B. 3-4x
- C. 12-20x
- D. 100x

Answer: C. 12-20x

Q16. PEFT mainly helps avoid which problem when data is small?

- A. Underfitting
- B. Overfitting / catastrophic forgetting
- C. Vanishing gradients
- D. Tokenization errors

Answer: B. Overfitting / catastrophic forgetting

Q17. Prompt tuning trains parameters at:

- A. Every transformer layer
- B. The input embedding layer only
- C. The attention heads only
- D. The output softmax only

Answer: B. The input embedding layer only

Q18. Prompt-tuning gains on T5 tend to plateau at roughly how many prompt tokens?

- A. 5
- B. 20
- C. 100
- D. 1000

Answer: B. 20

Q19. Prefix tuning differs from prompt tuning because it adds trainable params at:

- A. Only the input
- B. Every transformer layer
- C. Only the final layer
- D. The tokenizer

Answer: B. Every transformer layer

Q20. A key DISADVANTAGE of adapters vs LoRA is:

- A. More storage
- B. Added inference latency (in-series layers)
- C. Cannot reach good accuracy
- D. Requires no training

Answer: B. Added inference latency (in-series layers)

Q21. In LoRA, the update is written as:

- A. $W + xI$
- B. B.A added in parallel to frozen W
- C. A softmax over W
- D. W times a scalar

Answer: B. B.A added in parallel to frozen W

Q22. In LoRA, matrices are initialised so that at start the model is unchanged. This means:

- A. $A=0$, $B=0$
- B. $B=0$ (so $BA=0$), A =small Gaussian
- C. A =identity, B =identity
- D. Both random large

Answer: B. $B=0$ (so $BA=0$), A =small Gaussian

Q23. LoRA has no nonlinearity mainly so that:

- A. It trains slower
- B. B.A can be fused into W (no inference overhead)
- C. It can add layers in series
- D. It uses more memory

Answer: B. B.A can be fused into W (no inference overhead)

Q24. Which four matrices does LoRA typically adapt?

- A. Only feed-forward
- B. Query, Key, Value, Output projections
- C. Embedding and unembedding
- D. LayerNorm scales

Answer: B. Query, Key, Value, Output projections

Q25. The rank r in LoRA is conceptually synonymous with:

- A. Batch size
- B. Intrinsic dimensionality
- C. Learning rate
- D. Vocabulary size

Answer: B. Intrinsic dimensionality

Q26. BF16 compared to FP16 keeps the same:

- A. Mantissa bits
- B. Exponent range (dynamic range)
- C. Sign handling only
- D. Storage as INT8

Answer: B. Exponent range (dynamic range)

Q27. For symmetric INT8 quantization with max-abs value alpha, the scale factor is:

- A. $\alpha/127$
- B. $127/\alpha$
- C. $255/\alpha$
- D. $\alpha*127$

Answer: B. $127/\alpha$

Q28. Post-training quantization estimates activation ranges using a:

- A. Full retraining
- B. Calibration dataset
- C. Random init
- D. Larger model

Answer: B. Calibration dataset

Q29. Above ~2.7B parameters, plain INT8 degrades because of:

- A. Too few layers
- B. Outlier activation values
- C. Small vocabulary
- D. Missing embeddings

Answer: B. Outlier activation values

Q30. LLM.int8() handles outliers by:

- A. Deleting them
- B. Keeping outlier dimensions in FP16 and quantizing the rest
- C. Rounding them to zero
- D. Doubling the bits everywhere

Answer: B. Keeping outlier dimensions in FP16 and quantizing the rest

Q31. In QLoRA, NF4 assumes weights follow which distribution?

- A. Uniform
- B. Normal (Gaussian)
- C. Exponential
- D. Bernoulli

Answer: B. Normal (Gaussian)

Q32. In QLoRA, double quantization primarily saves:

- A. Accuracy
- B. Storage (quantizes the quantization constants)
- C. Latency
- D. Bandwidth only

Answer: B. Storage (quantizes the quantization constants)

Q33. QLoRA can fine-tune a 65B model in approximately:

- A. 780 GB
- B. 480 GB
- C. 48 GB
- D. 8 GB

Answer: C. 48 GB

Q34. Unstructured magnitude pruning gives no real speed-up unless:

- A. The model is small
- B. The hardware supports sparsity
- C. You retrain fully
- D. You use FP32

Answer: B. The hardware supports sparsity

Q35. The 2:4 sparsity pattern is an example of:

- A. Unstructured pruning
- B. Structured pruning
- C. Quantization
- D. Distillation

Answer: B. Structured pruning

Q36. Knowledge distillation trains the student to match the teacher's:

- A. Hard label only
- B. Output distribution (soft targets)
- C. Weights directly
- D. Gradients

Answer: B. Output distribution (soft targets)

Q37. The biggest practical bottleneck for distillation is:

- A. GPU speed
- B. Availability of training data
- C. Model size
- D. Tokenizer choice

Answer: B. Availability of training data

Q38. Among quantization, pruning, distillation, which is least lossy but most expensive?

- A. Quantization
- B. Pruning
- C. Distillation
- D. All equal

Answer: C. Distillation

Q39. Self-Instruct generates fine-tuning data starting from about how many seed tasks?

- A. 17
- B. 175
- C. 1750
- D. 17500

Answer: B. 175

Q40. In the residual-stream view, attention and MLP outputs are:

- A. Multiplied into the stream
- B. Added to the stream
- C. Concatenated then truncated
- D. Ignored

Answer: B. Added to the stream

Q41. The QK circuit of attention determines:

- A. What is written to the stream
- B. Where to attend (attention pattern)
- C. The vocabulary
- D. The learning rate

Answer: B. Where to attend (attention pattern)

Q42. The OV circuit is responsible for:

- A. Choosing attention weights
- B. Writing information to the residual stream
- C. Tokenization
- D. Normalization

Answer: B. Writing information to the residual stream

Q43. The unembedding matrix W_U maps:

- A. Tokens to embeddings
- B. Residual vector to vocabulary distribution
- C. Weights to gradients
- D. Images to text

Answer: B. Residual vector to vocabulary distribution

Q44. Probing evaluates a layer by:

- A. Deleting it
- B. Training a classifier on its hidden states for a property
- C. Quantizing it
- D. Pruning it

Answer: B. Training a classifier on its hidden states for a property

Q45. Selectivity in probing is defined as:

- A. Probe accuracy + control accuracy
- B. Probe-task accuracy - control-task accuracy
- C. Control accuracy only
- D. Random baseline

Answer: B. Probe-task accuracy - control-task accuracy

Q46. A control task uses ____ labels to check if the probe is just memorizing.

- A. Gold
- B. Random
- C. Averaged
- D. Weighted

Answer: B. Random

Q47. A neuron encoding several unrelated concepts is called:

- A. Monosemantic
- B. Polysemantic
- C. Sparse
- D. Dead

Answer: B. Polysemantic

Q48. Mechanistic interpretability aims to ____ the network's algorithm.

- A. Compress
- B. Reverse-engineer
- C. Quantize
- D. Prune

Answer: B. Reverse-engineer

Q49. A 'circuit' in interpretability is a:

- A. Full model
- B. Computational subgraph responsible for a task
- C. Loss function
- D. Dataset

Answer: B. Computational subgraph responsible for a task

Q50. Activation patching measures importance via the change in:

- A. Accuracy only
- B. Logit difference between clean and corrupt runs
- C. Parameter count
- D. Memory use

Answer: B. Logit difference between clean and corrupt runs

Q51. Attribution patching reduces cost by replacing per-component forward passes with:

- A. More forward passes
- B. A single backpropagation (linear approximation)
- C. Random sampling
- D. Quantization

Answer: B. A single backpropagation (linear approximation)

Q52. An induction head implements which behaviour?

- A. Token deletion
- B. Pattern copying ($[A][B] \dots [A] \rightarrow [B]$)
- C. Positional encoding
- D. Layer normalization

Answer: B. Pattern copying ($[A][B] \dots [A] \rightarrow [B]$)

Q53. Induction heads require at least how many layers to form?

- A. 1
- B. 2
- C. 10
- D. 24

Answer: B. 2

Q54. The in-context-learning score is computed as:

- A. $\text{loss@50} - \text{loss@500}$
- B. $\text{loss@500} - \text{loss@50}$
- C. loss@1 only
- D. accuracy@500

Answer: B. $\text{loss@500} - \text{loss@50}$

Q55. The logit lens works by multiplying an intermediate state by:

- A. The embedding matrix
- B. The unembedding matrix W_U
- C. A random matrix
- D. The attention weights

Answer: B. The unembedding matrix W_U

Q56. Superposition means the model stores:

- A. Fewer features than dimensions
- B. Exactly as many features as dimensions
- C. More features than dimensions
- D. No features

Answer: C. More features than dimensions

Q57. Sparse autoencoders are trained with reconstruction loss plus a penalty on the:

- A. L2 norm of weights
- B. L1 norm of the sparse features
- C. Number of layers
- D. Batch size

Answer: B. L1 norm of the sparse features

Q58. Sparse autoencoders help produce ____ features.

- A. Polysemantic
- B. Monosemantic (interpretable)
- C. Random
- D. Compressed

Answer: B. Monosemantic (interpretable)

Q59. Patchscopes reduces to the logit lens when $M = M'$ and the patched layer is the:

- A. First layer
- B. Middle layer
- C. Last layer
- D. Embedding layer

Answer: C. Last layer

Top Short-Answer Questions

- Define a knowledge-graph triple and give an example.
- What do Wikidata prefixes Q and P denote?
- Why are knowledge graphs described as incomplete? Give the Freebase figure.
- Name the two approaches to KGQA and one pro/con of each.
- State three reasons to combine LLMs with KGs.
- Define MRR and compute it if the first correct answer is at rank 3.
- Difference between Hits@K and MRR.
- What is the local closed-world assumption?
- Write the TransE score and name its two main limitations.
- How does RotatE model a relation, and which limitation of TransE does it fix?
- Why do FB15K-237 and WN18RR exist?
- Why is full fine-tuning impractical (give memory + storage reasons)?
- What is catastrophic forgetting and how does PEFT reduce it?
- Contrast prompt tuning and prefix tuning.
- State the adapter structure and its main disadvantage.
- Write the LoRA update and its initialization scheme.
- Give the INT8 symmetric scale-factor formula.
- Explain the outlier problem and how LLM.int8() solves it.
- List the four components of QLoRA.
- Distinguish unstructured vs structured pruning.
- Why is distillation the most expensive compression method?
- What are the QK and OV circuits?
- Define probing and selectivity.
- What does an induction head do and why does it need 2 layers?
- What is superposition and how do sparse autoencoders address it?

Top Long-Answer Questions

- Explain KG completion end-to-end: scoring function, positive/negative sampling, softmax vs margin loss, and evaluation (Hits@K, MRR, filtered eval).
- Compare TransE and RotatE in detail: formulation, what relation patterns each can/can't model, and why RotatE is a superset.
- Discuss the trade-offs between in-context learning, full fine-tuning, and PEFT across memory, storage, accuracy, and forgetting.
- Describe the four PEFT techniques (prompt tuning, prefix tuning, adapters, LoRA), with the parameter count and inference-latency implications of each.
- Explain quantization fully: number formats (FP32/FP16/BF16/INT8), the scale-factor computation, PTQ vs QAT, the outlier problem, and LLM.int8().
- Explain QLoRA's four components and why it drastically reduces memory while preserving accuracy.

- Compare quantization, pruning, and distillation on how they work, lossiness, cost, and hardware requirements.
- Explain the residual-stream view of a transformer, the QK/OV circuit decomposition, and why it enables mechanistic interpretability.
- Compare activation patching and attribution patching: procedure, cost, and limitations.
- Explain induction heads and their link to in-context learning, and how superposition motivates sparse autoencoders.

Important Concepts — Ranked by Exam Probability

Concept	Probability
MRR / Hits@K definition & computation	HIGH
TransE vs RotatE (limitations)	HIGH
LoRA (formula, init, fusing, Q/K/V/O)	HIGH
PEFT vs full FT (memory 12–20×)	HIGH
INT8 scale factor = $127/\alpha$	HIGH
QLoRA four components	HIGH
Q vs P IDs; KG incompleteness (93.8%)	MEDIUM
Prompt vs prefix vs adapter distinctions	MEDIUM
Outliers & LLM.int8()	MEDIUM
Residual stream / QK & OV circuits	MEDIUM
Activation vs attribution patching	MEDIUM
Induction heads & ICL score	MEDIUM
Probing, selectivity, control tasks	MEDIUM
Superposition & sparse autoencoders	MEDIUM
Datasets (WN18/FB15K variants)	LOWER
Structured vs unstructured pruning specifics	LOWER
Self-Instruct / synthetic-data distillation	LOWER
Patchscopes / logit-lens details	LOWER

🔔 One-Day-Before-Exam Cram Card

- **KG**: triple (s,r,o); Q=entity, P=relation; incomplete (93.8% Freebase).
- **QA**: semantic parsing (interpretable) vs differentiable (end-to-end).
- **Metrics**: $MRR = 1/\text{rank}$; Hits@K; filtered removes known-true.
- **Models**: TransE $s+r \approx o$ (fails symmetric/1-to-many); RotatE rotation $|r|=1$ (superset).
- **PEFT**: full FT=12–20× mem; prompt(input)/prefix(all layers)/adapter(series,latency)/LoRA(BA,fuses).
- **LoRA**: $B=0, A=\text{Gaussian}$; Q/K/V/O; rank=intrinsic dim.
- **Quant**: $\text{scale} = 127/\alpha$; BF16 keeps range; PTQ vs QAT; outliers $\rightarrow \text{LLM.int8}()$.
- **QLoRA**: NF4 + double-quant + paged optimizers + LoRA (BF16 compute).
- **Prune**: unstructured (no HW speed-up) vs structured (2:4). **Distil**: soft>hard, costly, data-hungry.
- **Interp**: residual stream (QK where / OV write); probing+selectivity; activation vs attribution patching; induction heads $\rightarrow$ ICL; superposition $\rightarrow$ SAEs (wide+sparse, L1).